

# Hi3519DV500 eMMC 分区修改指南

## 1. 背景知识

### 1.1 eMMC 分区是如何生效的

Hi3519DV500 SDK 的 eMMC 分区定义涉及 3 个文件的协同：

emmc\_env.txt（分区定义源头）

|

|→ mkenvimage → emmc\_env.bin

|（u-boot 运行时读取，决定 bootcmd 如何加载各分区镜像）

|

|→ env2burn.py → emmc\_burn\_table.xml

|（烧写工具读取，决定烧写器如何分区及每个分区烧什么文件）

- emmc\_env.bin：二进制 u-boot 环境变量，由 mkenvimage 工具从文本文件生成，烧写到 eMMC env 分区
- emmc\_burn\_table.xml：XML 格式的烧写分区表，由 env2burn.py 脚本从同一个文本文件解析生成

两者都源自同一个文本文件，修改分区只需改文本文件，重新 make uboot\_env 即可同步更新两个产物。

### 1.2 相关文件清单

| 文件           | 路径（相对于 bsp/）                                         | 作用                              |
|--------------|------------------------------------------------------|---------------------------------|
| emmc_env.txt | tools/pc/uboot_env/env_text/hi3519dv500/emmc_env.txt | u-boot 环境变量文本源（分区定义源头）          |
| env2burn.py  | tools/pc/uboot_env/scripts/env2burn.py               | 从 env 文本解析生成 burn_table.xml     |
| Makefile     | Makefile                                             | EMMC_ROOTFS_SIZE 变量控制 ext4 镜像大小 |
| Makefile     | Makefile                                             | EXT4_IMAGE_BIN 变量控制 ext4 镜像文件名  |

### 1.3 编译命令

cd /workspace/.../bsp

make uboot\_env # 重新生成 emmc\_env.bin + emmc\_burn\_table.xml

make rootfs\_build # 重新打包 ext4 镜像（仅改大小时需要）

## 2. 当前分区布局

emmc\_env.txt 中的分区定义：

blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)

| 分区   | 偏移             | 大小    | 设备节点           | 烧写文件           |
|------|----------------|-------|----------------|----------------|
| boot | 0x000000 (0MB) | 512KB | /dev/mmcblk0p1 | boot_image.bin |

| 分区       | 偏移                  | 大小    | 设备节点           | 烧写文件              |
|----------|---------------------|-------|----------------|-------------------|
| env      | 0x080000 (512KB)    | 512KB | /dev/mmcblk0p2 | emmc_env.bin      |
| bl31     | 0x100000 (1MB)      | 512KB | /dev/mmcblk0p3 | bl31.bin          |
| kernel   | 0x180000 (1.5MB)    | 13MB  | /dev/mmcblk0p4 | ulmage-fdt        |
| rootfs   | 0x2F0000 (14.5MB)   | 200MB | /dev/mmcblk0p5 | rootfs_ext4.img   |
| userdata | 0x18F0000 (214.5MB) | 50MB  | /dev/mmcblk0p6 | userdata_ext4.img |

### 3. 修改场景教程

#### 场景 A：调整现有分区大小

以将 rootfs 从 96MB 改为 200MB 为例。

##### 步骤 1：修改 Makefile 中的 ext4 镜像大小

文件： `bsp/Makefile`

```
# 修改前
EMMC_ROOTFS_SIZE ?= 96

# 修改后
EMMC_ROOTFS_SIZE ?= 200
```

此变量控制 `rootfs_build` 目标生成的 ext4 镜像大小（dd 创建空文件的字节数）。

##### 步骤 2：修改 emmc\_env.txt 中的分区表

文件： `bsp/tools/pc/uboot_env/env_text/hi3519dv500/emmc_env.txt`

找到 `bootargs` 行中的 `blkdevparts=` 部分：

```
# 修改前
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),96M(rootfs)

# 修改后
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs)
```

**注意：**分区大小支持的单位为 K、M、G（如 `200M`）。不支持 `-`（剩余空间）语法。

##### 步骤 3：重新编译

```
make uboot_env      # 更新 emmc_env.bin + emmc_burn_table.xml
make rootfs_build    # 重新生成 200MB 的 ext4 镜像
```

## 验证

检查 `pub/hi3519dv500_emmc_image_musl/` 下：

- `emmc_env.bin` 更新时间是否为刚刚
- `emmc_burn_table.xml` 中 `rootfs` 分区的 `Length` 和 `SelectFile` 是否正确

## 场景 B：修改镜像文件名

以将 `rootfs` 镜像名从 `rootfs_hi3519dv500_200M.ext4` 改为 `rootfs_ext4.img` 为例。

### 步骤 1：修改 Makefile 中的镜像文件名变量

文件： `bsp/Makefile`

```
# 修改前
EXT4_IMAGE_BIN:=rootfs_${(CHIP)}_${(EMMC_ROOTFS_SIZE)}M.ext4

# 修改后
EXT4_IMAGE_BIN:=rootfs_ext4.img
```

此变量控制编译产物的文件名（`dd`、`mkfs.ext4`、`populate-extfs.sh` 均引用此变量）。

### 步骤 2：修改 env2burn.py 中的文件名映射

文件： `bsp/tools/pc/uboot_env/scripts/env2burn.py`

找到 `select_rootfs` 函数中 `flash_type == 'emmc'` 分支：

```
# 修改前
if flash_type == 'emmc':
    return 'rootfs_%s_96M.ext4' % plat

# 修改后
if flash_type == 'emmc':
    return 'rootfs_ext4.img'
```

此函数的返回值是 `emmc_burn_table.xml` 中 `rootfs` 分区的 `<SelectFile>` 字段值，烧写工具靠这个文件名去匹配实际的镜像文件。

### 步骤 3：重新编译

```
make uboot_env      # burn_table.xml 中的 SelectFile 字段更新
make rootfs_build    # 生成新文件名的 ext4 镜像
```

## 场景 C：新增分区

以在 rootfs 后新增 50MB userdata 分区为例。

### 步骤 1：修改 emmc\_env.txt 追加分区

文件： `bsp/tools/pc/uboot_env/env_text/hi3519dv500/emmc_env.txt`

在 `blkdevparts=` 的分区列表末尾追加：

```
# 修改前
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs)

# 修改后
blkdevparts=mmcblk0:512K(boot),512K(env),512K(bl31),13M(kernel),200M(rootfs),50M(userdata)
```

分区之间用英文逗号 `,` 分隔。格式为 `大小(名称)`。

**注意：** `env2burn.py` 不支持 `-`（剩余空间）语法，必须指定明确大小。

### 步骤 2：修改 env2burn.py 注册新分区的烧写文件名

文件： `bsp/tools/pc/uboot_env/scripts/env2burn.py`

找到 `select_file` 函数，在 `rootfs` 分支之后、`return ''` 之前添加新分支：

```
def select_file(flash_type, part):
    if part.name == 'boot':
        return select_boot(part.name)
    if part.name == 'env':
        return select_env(flash_type)
    if part.name in ['bl31', 'teeimg']:
        return select_fw(part.name)
    if part.name == 'kernel':
        return select_kernel()
    if part.name == 'rootfs':
        return select_rootfs(flash_type, part.size)
    # ---- 新增 ----
    if part.name == 'userdata':
        return 'userdata_ext4.img'
    # ---- 新增结束 ----
    return ''
```

如果不添加此分支，`userdata` 分区在 `burn_table.xml` 中的 `SelectFile` 字段为空，烧写时不会向该分区写入文件（仅创建空分区）。如果这正是你想要的，可以跳过此步骤。

### 步骤 3：重新编译

```
make uboot_env      # 更新分区表（包含新的 userdata 分区）
```

## 场景 D：调整 Linux 可见内存

修改 `emmc_env.txt` 中 `bootargs` 的 `mem=` 参数：

```
# 修改前
bootargs=mem=132928K ...

# 修改为 256MB
bootargs=mem=262144K ...

# 修改为 512MB
bootargs=mem=524288K ...
```

`mem=` 值不能超过物理 DDR 扣除 MPP/ISP 硬件保留后的可用内存。设置过大会导致内核启动崩溃。

```
make uboot_env      # 更新 emmc_env.bin
```

## 4. emmc\_env.txt 完整参数说明

|                                     |                                                        |
|-------------------------------------|--------------------------------------------------------|
| <code>arch=arm</code>               | # CPU 架构                                               |
| <code>baudrate=115200</code>        | # 串口波特率                                                |
| <code>board=hi3519dv500</code>      | # 板级标识                                                 |
| <code>board_name=hi3519dv500</code> | # 板子名称                                                 |
| <code>bootdelay=2</code>            | # u-boot 启动倒计时（秒），按任意键可进入命令行                           |
| <code>cpu=armv8</code>              | # CPU 内核版本                                             |
| <code>ethact=gmac0</code>           | # 默认以太网控制器                                             |
| <code>loadaddr=0x40080000</code>    | # 网络下载默认加载地址                                           |
| <code>soc=hi3519dv500</code>        | # SoC 型号（ <code>env2burn.py</code> 用于确定镜像文件名中的平台标识）    |
| <code>stderr=serial</code>          | # 标准错误 → 串口                                            |
| <code>stdin=serial</code>           | # 标准输入 ← 串口                                            |
| <code>stdout=serial</code>          | # 标准输出 → 串口                                            |
| <code>vendor=vendor</code>          | # 厂商标识                                                 |
| <code>verify=n</code>               | # 启动时不校验内核镜像（ <code>y=校验</code> ， <code>n=不校验</code> ） |
| <code>bootargs=...</code>           | # Linux 内核启动参数（见下方详细说明）                                |
| <code>bootcmd=...</code>            | # u-boot 自动启动命令（见下方详细说明）                               |

### bootargs 参数详解

```
bootargs=mem=132928K console=ttyAMA0,115200 clk_ignore_unused root=/dev/mmcblk0p5
rootfstype=ext4 rw rootwait
blkdevparts=mmcblk0:512K(boot),512K(env),512K(b131),13M(kernel),200M(rootfs),50M(userdata)
```

| 参数                                  | 说明                    |
|-------------------------------------|-----------------------|
| <code>mem=132928K</code>            | Linux 可见内存大小（扣除硬件保留后） |
| <code>console=ttyAMA0,115200</code> | 内核控制台串口及波特率           |

| 参数                                   | 说明                         |
|--------------------------------------|----------------------------|
| <code>clk_ignore_unused</code>       | 不关闭"未使用"时钟（防止 MPP 模块异常）    |
| <code>root=/dev/mmcblk0p5</code>     | 根文件系统设备节点（第 5 分区 = rootfs） |
| <code>rootfstype=ext4</code>         | 根文件系统类型                    |
| <code>rw</code>                      | 读写模式挂载                     |
| <code>rootwait</code>                | 等待 eMMC 就绪后再挂载             |
| <code>blkdevparts=mmcblk0:...</code> | eMMC 分区表定义                 |

bootcmd 命令详解

```
bootcmd=mmc read 0 0x40010000 0x800 0x400; ipcboot bl31 0x40010000; mmc read 0 0x44000000 0xc00 0x6800; ipcboot uimgfdt 0x44000000
```

| 命令                                              | 说明                                                     |
|-------------------------------------------------|--------------------------------------------------------|
| <code>mmc read 0 0x40010000 0x800 0x400</code>  | 从 eMMC 扇区 0x800 读 0x400 扇区（bl31 分区）到 DDR 0x40010000    |
| <code>ipcboot bl31 0x40010000</code>            | 启动 ARM Trusted Firmware（海思自定义命令）                       |
| <code>mmc read 0 0x44000000 0xc00 0x6800</code> | 从 eMMC 扇区 0xc00 读 0x6800 扇区（kernel 分区）到 DDR 0x44000000 |
| <code>ipcboot uimgfdt 0x44000000</code>         | 启动 Linux 内核（ulmage+设备树）                                |

5. 修改后完整验证流程

5.1 编译验证

```
cd /workspace/.../bsp
make uboot_env
```

确认无报错，且输出：

```
Generate env.bin
Generate burn_table.xml
```

## 5.2 检查 burn\_table.xml

```
cat pub/hi3519dv500_emmc_image_musl/emmc_burn_table.xml
```

确认：

- 分区数量正确
- 每个分区的 `Start`、`Length` 偏移正确
- `SelectFile` 字段与实际镜像文件名一致

## 5.3 检查编译产物

```
ls -la pub/hi3519dv500_emmc_image_musl/
```

确认 `emmc_env.bin`、`emmc_burn_table.xml` 的修改时间为刚刚。

## 5.4 烧写验证

将 `emmc_env.bin`、`emmc_burn_table.xml` 连同其他镜像一起烧写到板子，启动后：

```
# 在 u-boot 命令行（按任意键打断启动）
=> printenv bootargs      # 确认分区表
=> printenv bootcmd       # 确认启动命令

# 内核启动后
# cat /proc/partitions    # 确认分区节点
# df -h                  # 确认 rootfs 挂载大小
# cat /proc/meminfo       # 确认内存大小
```

## 6. 常见问题

### Q1: 修改分区后 `make uboot_env` 报错 `wrong partition size`

**原因：**分区大小格式不正确。`env2burn.py` 仅支持 `数字+单位(名称)` 格式，不支持 `-`（剩余空间）。

**解决：**使用明确大小，如 `50M(userdata)`，不要用 `-(userdata)`。

### Q2: 烧写后 `rootfs` 挂载失败

**原因：**`burn_table.xml` 中 `rootfs` 的 `SelectFile` 与实际文件名不匹配。

**解决：**检查 `env2burn.py` 的 `select_rootfs` 函数返回值是否与 `Makefile` 中 `EXT4_IMAGE_BIN` 一致。

### Q3: 内核启动后看不到新分区

**原因：**`blkdevparts=` 修改后只更新了 `env`，但没有重新烧写 `emmc_env.bin`。

**解决：**重新烧写 `emmc_env.bin` 到 eMMC 的 `env` 分区。

## Q4: 修改 mem= 后内核启动崩溃

**原因：** mem= 值超过了物理 DDR 扣除硬件保留后的可用内存。

**解决：** 调小 mem= 值，或先在 u-boot 命令行用 bdinfo 确认实际可用内存范围后再设置。